

Vyaha / NearU Project Functionality Guide

Generated: 11 June 2026

This document explains the current project in depth based on the code in this repository. The workspace is named NearU, while the product branding inside the apps and website is Vyaha.

1. Project Overview

Vyaha is a hyperlocal ordering and delivery platform. It is not a single app; it is a complete ecosystem with separate tools for each side of the business:

- Customer mobile app: lets customers sign in, discover nearby shops/restaurants, browse menus, manage cart and addresses, pay or choose cash on delivery, track orders, view history, and contact support.
- Partner mobile app: lets restaurant/shop owners sign in, complete business onboarding, upload verification documents, manage shop profile, manage menus, receive and update orders, and view payout/payment history.
- Delivery mobile app: lets riders sign in, complete delivery partner profile and document verification, go available, view nearby ready jobs, accept or reject jobs, update pickup and delivery status, track earnings, and manage cash collected from COD orders.
- Admin web panel: internal operations portal for admins to review partners and delivery partners, manage orders, assign delivery, handle support, create payouts, and verify cash deposits.
- Public website: marketing, policy, support, legal, fraud reporting, partner, delivery, and app release information.
- Backend API: Express/MongoDB service that connects all apps, owns authentication, users, partners, menus, orders, payment verification, delivery jobs, uploads, notifications, support, payouts, and admin operations.

The main business idea is local-first ordering: customers see approved nearby shops, order at transparent totals, restaurants manage their own menu and order status, and delivery partners handle the final delivery workflow.

2. Repository Layout

The important top-level project areas are:

- apps/customer-app: Expo React Native customer app.
- apps/partner-app: Expo React Native restaurant/shop partner app.
- apps/delivery-app: Expo React Native delivery partner app.
- apps/admin-panel: Vite React TypeScript admin web panel.
- backend: Node.js Express TypeScript API.

- vyaha-official: Vite React public website.
- docs: project documentation, QA checklist, Play Store readiness notes, and this guide.
- plugins: custom Expo config plugins used by the mobile apps.

Each app is independently runnable and has its own package.json. The backend is also independently runnable. There is no root-level package file currently visible in the workspace.

3. Technology Stack

Mobile Apps

The customer, partner, and delivery apps are built with:

- Expo SDK 54.
- React Native 0.81.x.
- React 19.1.x.
- TypeScript.
- React Navigation for stack and tab navigation.
- Axios/fetch based API clients.
- AsyncStorage for local session storage.
- Firebase native modules for app initialization, phone auth, and FCM messaging.
- Expo Location for customer address pinning, nearby shop lookup, shop location pinning, and rider location/distance support.
- Expo Image Picker and Document Picker in partner/delivery apps for document and image uploads.
- Razorpay React Native SDK in the customer app for online payments.
- Custom Expo plugins for Android Gradle configuration and Android notification channel/icon setup.
- EAS build/update configuration through app.json and eas.json where present.

Web Apps

The admin panel is built with:

- Vite.
- React 19.
- TypeScript.
- React Router.
- Ant Design.
- Axios.

The public website is built with:

- Vite.
- React.
- React Router.
- Lucide React icons.

Backend

The backend is built with:

- Node.js 20.
- Express 5.
- TypeScript.
- MongoDB with Mongoose.
- JWT access and refresh tokens.
- Firebase Admin SDK for Firebase phone token verification and FCM push notifications.
- Razorpay server SDK and HMAC signature verification.
- Cloudinary for uploads.
- Express file upload for multipart image/document upload.
- Helmet, CORS, and rate limiting.
- Jest, Supertest, and MongoDB memory server for tests.

4. Runtime Applications

Customer App: apps/customer-app

The customer app is the buyer-facing app named Vyaha. Its Android package is com.vyaha.customer. It supports location and notifications, and uses Firebase and Razorpay.

Main screens:

- Login: phone number login entry.
- Otp: OTP or Firebase phone verification completion.
- Home: nearby shop discovery.
- ShopDetail: shop menu and add-to-cart.
- Cart: cart review, address selection, location pin capture, and pricing quote.
- Payment: COD or Razorpay checkout.
- OrderStatus: detailed order progress tracking.
- Orders: order history.
- Profile: profile, saved addresses, recent orders, support, logout, delete account.
- OrderSummary: older/intermediate checkout summary screen still present in code.

The app starts by checking AsyncStorage for a token. If no token exists, the user sees login. If a token exists, it fetches the backend profile and checks whether the customer has a real name and a complete pinned address. Customers without required profile data are forced to complete profile before reaching home.

Profile completeness is based on:

- Name must not be a generated default like Customer.
- Name length must be at least 3.
- Address must include house/door or street, road/street, city, state, pincode, area/locality.

- Address must include a real latitude/longitude pin.

Important customer functionality:

- Phone OTP login with role customer.
- Firebase phone token verification support.
- API fallback to production and local development URLs.
- Automatic access token refresh using refresh tokens.
- Nearby shop lookup using foreground location.
- Fallback to approved shops if location permission is denied or geo lookup fails.
- Public menu browsing for selected shops.
- Cart management through React Context.
- Server-side pricing quote before checkout.
- Exact delivery pin required before placing orders.
- COD and Razorpay payment options.
- Order tracking with status and payment status descriptions.
- Saved addresses and default address management.
- Recent order display in profile.
- Support FAQs and ticket flow.
- Logout and account deletion.

Partner App: apps/partner-app

The partner app is for restaurants and shops. Its Android package is com.vyaha.partner. It has a blue partner-branded UI, Firebase messaging, location permission for shop pinning, and image/document upload permissions.

Main screens:

- Login: partner phone number login.
- Otp: OTP/Firebase verification.
- Onboarding: restaurant/shop registration and document upload.
- ApplicationSubmitted: success screen after onboarding submission.
- PendingApproval: waiting for admin verification.
- Rejected: rejection/reupload state.
- WelcomeApproved: approved partner onboarding bridge.
- Dashboard: main operational home.
- Orders: partner order list.
- OrderDetails: accept/reject and preparation flow.
- Menu: menu item management.
- Profile: shop profile, images, documents, reupload handling.
- Settings: shop settings and account actions.
- PaymentHistory: wallet/payout/payment history area.

The app checks stored token and phone, refreshes the session, then calls /partners/my-status. Routing depends on partner status:

- No partner profile: send to onboarding.

- PENDING: show pending approval.
- APPROVED: show dashboard.
- REJECTED: show rejected screen.
- SUSPENDED: return to login.

Important partner functionality:

- Phone OTP login with role partner.
- Partner onboarding with owner, restaurant, category, address, shop GPS pin, bank/document details.
- Draft saving for onboarding locally and remotely.
- Cloudinary upload through the backend upload endpoint.
- Admin document reupload flags and notes shown in profile.
- Shop profile editing: restaurant name/details, images, photos, hours, status, address, location, and documents.
- Shop open/closed toggle.
- Menu CRUD: create, edit, delete, search, category filtering, item image upload, vegetarian flag, prep time, availability toggle.
- First menu item can mark setup as complete.
- Order workflow: view confirmed orders, accept, reject, mark preparing, mark ready.
- Partner receives push notifications for new orders and delivery updates.
- Wallet/payment history for payout visibility.
- Logout and delete account.

Delivery App: apps/delivery-app

The delivery app is for riders. Its Android package is com.vyaha.delivery. It supports location, notifications, and document uploads.

Main screens:

- Login: delivery phone number login.
- Otp: OTP/Firebase verification.
- Main: bottom tab navigator.
- Jobs: available delivery jobs.
- MyJobs: assigned/current jobs.
- JobDetails: accept job, navigate pickup/drop, mark pickup/delivery.
- Earnings: daily earnings, stats, COD ledger, deposit submission.
- Profile: delivery profile and account management.
- CompleteProfile: forced delivery profile completion.
- ReviewStatus: verification/rejection/suspended/inactive states.
- AccountProfileScreen: extra account profile UI exists in code.

Routing depends on the delivery partner profile:

- ACTIVE or VERIFIED: main app.
- Missing/incomplete profile: complete profile.
- PENDING, REJECTED, SUSPENDED, or INACTIVE: review status.

Important delivery functionality:

- Phone OTP login with role delivery.
- Backend can auto-create an inactive delivery profile on delivery login if one does not exist.
- Profile completion with identity, vehicle, emergency, bank, and document fields.
- Admin verification status controls access to jobs.
- Available jobs list only shows eligible ready orders.
- Rider availability toggle.
- Foreground location capture and upload.
- Nearby job ranking by distance to restaurant when rider location exists.
- Accept and reject jobs.
- One active delivery guard: rider cannot accept another job while assigned/picked up order exists.
- Job details include pickup, drop, COD/prepaid information, earnings, and delivery status actions.
- Pickup status requires order to be assigned.
- Delivered status requires pickup first.
- COD orders require collected amount before delivery completion.
- Earnings are updated when deliveries complete.
- COD collected cash is tracked in a ledger.
- Cash deposit submission lets rider report money deposited back to platform for admin verification.
- Logout and account deletion.

Admin Panel: apps/admin-panel

The admin panel is an internal operations website. It uses admin password login, stores adminToken in local storage, and redirects to login on unauthorized responses.

Main pages:

- /login: admin password login.
- /: dashboard.
- /partners: partner verification and management.
- /delivery-partners: delivery partner verification and management.
- /orders: all order listing.
- /orders/:orderId: order detail and operational actions.
- /payouts: partner and delivery payout creation/history and cash deposit management.
- /support: support ticket queue and replies.

Important admin functionality:

- Login through /auth/admin-password-login.
- Dashboard stats for orders, partners, active partners, pending partners, earnings, and today's activity.
- Partner review: approve, reject, suspend, request document reupload.
- Delivery partner review: verify, activate, reject, suspend, request reupload.

- Order management: view all orders, view order details, update admin-supported status, assign delivery partner.
- Support management: list tickets, reply, change ticket status.
- Payout management: generate payout summaries by weekly/monthly periods, create paid payout records, view payout history.
- COD cash deposit management: list deposits, verify deposits, reject deposits.

Public Website: vyaha-official

The public website is the marketing and legal front door for Vyaha.

Main routes:

- /: marketing homepage.
- /policies: policy hub.
- /about: product and company overview.
- /blog: placeholder content/update section.
- /partner: partner onboarding information.
- /fraud: fraud reporting information.
- /support: support contact page.
- /privacy: privacy policy.
- /security: security policy.
- /terms: terms of service.
- /refunds: cancellation and refund policy.
- /community-guidelines: community and content guidelines.
- /partner-policy: restaurant partner policy.
- /delivery-policy: delivery partner policy.
- /cookie-policy: cookie policy.
- /delete-account: account deletion information.
- /restaurants, /apps, /consulting, /delivery: product-specific landing pages.

The website describes Vyaha as a hyperlocal food delivery platform focused on genuine prices, transparent totals, local restaurants, and separate tools for customers, restaurants, and riders.

5. Backend Architecture

The backend Express app is created in `backend/src/app.ts` and started in `backend/src/server.ts`.

Core middleware:

- CORS with configured production origins and local development allowance.
- Helmet for baseline HTTP hardening.
- Raw body parsing only for Razorpay webhook route so webhook signatures can be verified correctly.
- JSON body parsing for normal API routes.
- Rate limiting for general API traffic.

- Separate stricter rate limiting for OTP and admin login routes.
- Central error middleware.

Mounted route groups:

- /api/auth: OTP, Firebase token verification, admin login, refresh token, logout.
- /api/users: customer profile, addresses, customer orders, account deletion.
- /api/partners: partner onboarding, partner status, partner profile, shop status, menu subroutes, partner admin routes, public shop list.
- /api/menu: public partner menu and protected partner menu management.
- /api/orders: customer order creation/pricing/history, partner status updates, delivery job flow, admin delivery assignment.
- /api/admin: admin dashboard, orders, partners, support, payouts, cash deposits.
- /api/delivery: delivery profile, stats, earnings, jobs, location, cash ledger.
- /api/upload: Cloudinary-backed upload and deletion.
- /api/payment: Razorpay checkout order creation, payment verification, webhook.
- /api/support: customer support FAQs and tickets.
- /api/notifications: FCM token registration and removal.
- /legal: backend-served legal URLs for Play Store and public compliance.
- /health: simple health endpoint.

On server startup:

- Environment is validated.
- MongoDB connects.
- Stale unaccepted orders are auto-cancelled immediately.
- A timer runs stale-order cancellation every minute.
- Express listens on configured port.

6. Authentication and Sessions

The project uses phone-based authentication for mobile roles and password-based login for the admin panel.

Roles:

- customer
- partner
- delivery
- admin

The backend also defines `CONSUMER_APP_ROLES` as customer, partner, and delivery, meaning partner and delivery accounts may use customer-ordering routes too.

OTP flow:

1. App calls `/api/auth/send-otp` with phone and role.
2. Backend validates phone and role.
3. OTP is sent through configured provider.

4. App calls `/api/auth/verify-otp` with phone, role, and either OTP or Firebase ID token.
5. Backend verifies OTP/provider token.
6. Backend creates user if needed, except admin must already be valid/restricted.
7. Backend returns access token, refresh token, user profile, role, partner ID if linked, and delivery partner ID if linked.

Supported OTP providers:

- Twilio Verify.
- MSG91.
- In-memory OTP for development only.
- Firebase phone ID token verification.

Token behavior:

- Access tokens are short-lived JWTs.
- Refresh tokens are longer-lived JWTs.
- Token payload includes user ID, phone, role, name, partner ID, delivery partner ID, and sessionVersion.
- sessionVersion lets logout/account deletion invalidate existing tokens.
- Mobile/web clients retry expired-token requests by calling `/api/auth/refresh`.

Admin login:

- Admin panel calls `/api/auth/admin-password-login`.
- Backend requires `ADMIN_PANEL_PHONE` and `ADMIN_PANEL_PASSWORD`.
- If the admin user does not exist, backend creates it.
- Admin receives the same token style but with role admin.

Logout:

- `/api/auth/logout` increments sessionVersion.
- Optional notification token is removed from the user's token list.
- Apps clear local storage.

Account deletion:

- `/api/users/me` deactivates/anonymizes account data.
- Active sessions are invalidated by session version changes.
- Some order/payout/legal records remain where required.

7. Data Model

User

User is the shared account model for customers, partners, delivery partners, and admins.

Important fields:

- phone: unique account identifier.
- name, email, role.

- address: current/default customer address.
- addresses: saved customer addresses with labels and GPS pins.
- notificationTokens: FCM tokens by app and platform.
- partnerOnboardingDraft: saved partner onboarding draft.
- isActive: account active/deactivated flag.
- sessionVersion: invalidates tokens when changed.
- lastLogin.

Partner

Partner represents a restaurant/shop.

Important fields:

- Owner and restaurant/shop details.
- Phone and email.
- Shop images and banner images.
- Structured address and Google Maps link.
- GeoJSON shop location for nearby search and routing.
- Category such as restaurant, bakery, grocery, fast-food, sweets, ice-creams, juice, etc.
- User linkage through userId.
- Shop open status, opening time, closing time, weekly holidays.
- Rating.
- Verification documents: FSSAI, PAN, Aadhaar, GST, shop license, bank proof, menu proof, address proof, restaurant photos.
- Document reupload flags and admin notes.
- Status: PENDING, APPROVED, REJECTED, SUSPENDED.
- Approval metadata and rejection reason.
- Setup completion and menu item count.
- Settings: auto accept orders, estimated prep time, delivery mode, self delivery partners, delivery radius, minimum order, UPI ID.
- Notification preferences.
- Language preference.

DeliveryPartner

DeliveryPartner represents rider profile and operations data.

Important fields:

- userId and phone.
- Name, email, DOB, address.
- Emergency contact.
- Vehicle type, vehicle number, license number.
- Profile photo and identity/vehicle/bank documents.
- Document reupload flags and admin notes.

- Availability.
- Status: PENDING, VERIFIED, ACTIVE, REJECTED, SUSPENDED, INACTIVE.
- Total deliveries and total earnings.
- COD cash balance and pending deposit amount.
- Current GeoJSON location.
- Rating and rating count.

MenuItem

MenuItem belongs to a partner.

Important fields:

- Name, description, price.
- Category.
- Image URL.
- Vegetarian flag.
- Preparation time.
- Availability.
- Partner ID.
- Rating.

Menu categories are constrained by shop type. For example, restaurants get categories such as meals, biryani, curries, rice, combos, hots, and other; bakeries get breads, cakes, pastries, cookies, puffs, buns; grocery gets staples, snacks, dairy, beverages, personal care, household; and so on.

Order

Order is the central operational model.

Important fields:

- orderType: currently always SHOP.
- Customer ID.
- Partner ID.
- Delivery partner user ID.
- Delivery rejected rider IDs.
- Self-delivery reservation state.
- Delivery address and exact delivery location.
- Items with menu item ID, name, quantity, price.
- Item total, delivery fee, GST fields, platform fee, tax discount, distance fields, grand total.
- Payment ID and Razorpay metadata.
- Payment method: RAZORPAY, CASH_ON_DELIVERY, CARD, UPI, WALLET.
- Payment status: PENDING, PAYMENT_PENDING_DELIVERY, PAID, FAILED, REFUNDED, CANCELLED.
- Order status: PENDING, CONFIRMED, ACCEPTED, PREPARING, READY, ASSIGNED, PICKED_UP, DELIVERED, CANCELLED, REJECTED.

- Cancellation reason and customer cancellation message.
- Delivery ready, auto-cancelled, and delivered timestamps.
- COD collection record.
- Partner payout and delivery payout settlement tracking.

Payout

Payout records admin-created payouts.

Important fields:

- Recipient type: partner or delivery partner.
- Recipient ID and snapshot.
- Weekly/monthly period.
- Amount, order count, order IDs.
- Bank snapshot.
- Payout breakdown for gross earnings, cash offset, net payable, cash due after offset.
- Paid reference, notes, paid date, paid admin.

CashLedgerEntry

CashLedgerEntry tracks COD and cash deposit accounting for delivery partners.

Entry types:

- COD_COLLECTED.
- EARNINGS_OFFSET.
- CASH_DEPOSIT_SUBMITTED.
- CASH_DEPOSIT_VERIFIED.

Statuses:

- POSTED.
- PENDING.
- VERIFIED.
- REJECTED.

SupportTicket

SupportTicket supports customer/admin support conversations.

Important fields:

- User ID.
- Optional order ID.
- Category: customer support, order, payment, delivery, account, report issue, other.
- Subject.
- Status: open, in progress, resolved, closed.
- Priority.
- Messages with sender role and timestamps.

8. Customer Ordering Flow

Customer order lifecycle:

1. Customer signs in with phone OTP.
2. App checks profile completeness.
3. Customer grants location permission or continues with fallback.
4. Home screen requests approved shops near the customer.
5. Customer opens a shop and fetches its public menu.
6. Customer adds menu items to cart.
7. Cart groups items by shop.
8. Cart requires delivery address and exact GPS pin.
9. Cart requests /orders/pricing to calculate delivery fee and totals.
10. Customer chooses COD or Razorpay.
11. Customer creates order through /orders.
12. If COD, order starts as CONFIRMED and partner is notified.
13. If Razorpay, order starts as PENDING.
14. Customer pays through Razorpay.
15. App verifies payment through /payment/verify.
16. Backend marks payment PAID, order CONFIRMED, and notifies partner.
17. Partner accepts/rejects/prepares/marks ready.
18. Delivery partner accepts job.
19. Delivery partner picks up and delivers.
20. Customer tracks status in OrderStatus.

Pricing logic:

- Shop GPS pin and customer delivery pin are required for accurate pricing.
- Distance is calculated with Haversine distance between shop and customer.
- First kilometer fee is Rs 15.
- Additional kilometer fee is Rs 10.
- Food GST, delivery GST, and platform fee fields are recorded, but current payable total is item total plus delivery fee because tax/fee offers are applied as discounts.
- Delivery fee is visible before order confirmation.

Order validation:

- Partner must exist.
- Partner must be approved.
- Partner must be open.
- Items must exist.
- Items must be available.
- Submitted item price must match server menu price.
- Delivery pin must exist.
- Payment method must be valid.

9. Partner Order Flow

Partner lifecycle:

1. Partner signs in with OTP.
2. If no partner profile exists, app opens onboarding.
3. Partner enters owner, restaurant, address, category, GPS, bank, and document details.
4. Files are uploaded through backend Cloudinary endpoint.
5. Onboarding is submitted to backend.
6. Admin reviews and approves/rejects.
7. Approved partner reaches dashboard.
8. Partner creates first menu item; setup can be marked complete.
9. Customer orders arrive as CONFIRMED.
10. Partner can accept or reject.
11. Accepted order moves to ACCEPTED.
12. Partner marks preparing.
13. Partner marks ready.
14. Ready order becomes available for delivery partners or reserved self-delivery riders.

Partner order statuses:

- CONFIRMED: order is paid/COD confirmed and waiting for restaurant.
- ACCEPTED: restaurant accepted.
- PREPARING: restaurant is preparing.
- READY: order is ready for pickup.
- REJECTED: restaurant rejected, order is cancelled/refund marked where applicable.

Important safeguards:

- Partner cannot accept unpaid online orders.
- Rejected orders are treated as cancelled with a customer-facing refund message.
- Terminal orders cannot be updated again.
- Only the partner that owns the order, or admin, can update partner-side status.
- Ready orders trigger delivery job notifications.

10. Delivery Flow

Delivery partner lifecycle:

1. Rider signs in with phone OTP.
2. Backend links or creates delivery partner profile.
3. Rider completes profile and document upload.
4. Admin reviews and verifies/activates.
5. Rider can enter main app when VERIFIED or ACTIVE.
6. Rider toggles availability.
7. App captures foreground location.

8. Backend stores rider location and may promote verified riders to active after valid location sharing.
9. Rider sees ready jobs.
10. Jobs are ranked by distance to restaurant when rider location is available.
11. Rider accepts one job.
12. Order becomes ASSIGNED.
13. Rider marks pickup.
14. Order becomes PICKED_UP.
15. Rider completes delivery.
16. Order becomes DELIVERED.
17. Earnings and COD ledger update.

Delivery job safeguards:

- Only VERIFIED or ACTIVE riders can see/accept jobs.
- Rider must be available.
- Rider cannot accept a second active delivery.
- Ready jobs expire if not accepted within the configured delivery window.
- Rider can reject jobs so the same job is not shown again to that rider.
- For COD, delivery cannot be completed until collected amount is at least the order grand total.

COD accounting:

- COD collected on delivery is recorded as COD_COLLECTED.
- Rider cashBalance increases.
- Rider can submit a cash deposit with amount, reference, proof, and note.
- Admin can verify or reject deposit.
- Verified deposits reduce rider cash balance.
- Payout calculations offset delivery earnings against cash held where needed.

11. Payment Flow

Payment methods supported by backend model:

- Razorpay.
- Cash on delivery.
- Card, UPI, Wallet are present as enum values, but the main customer UI uses Razorpay and COD.

Razorpay flow:

1. Customer creates an order.
2. Customer app calls /payment/create-order.
3. Backend creates Razorpay order for grandTotal * 100 paise.
4. Backend stores Razorpay order ID and returns Razorpay key ID to the app.
5. App opens Razorpay checkout.
6. App receives Razorpay payment result.

7. App calls /payment/verify.
8. Backend verifies HMAC checkout signature.
9. If valid, backend marks order PAID and CONFIRMED.
10. Backend notifies partner of new order.

Webhook flow:

- /payment/webhook uses raw JSON body.
- Backend verifies x-razorpay-signature.
- payment.captured marks order paid and confirmed.
- payment.failed marks order payment failed.
- Webhook can also notify partner when payment confirms.

COD flow:

- Customer creates order with CASH_ON_DELIVERY.
- Backend sets payment status PAYMENT_PENDING_DELIVERY.
- Backend sets order status CONFIRMED.
- Partner can accept COD orders immediately.
- Delivery partner must collect cash before marking delivered.
- Backend marks payment PAID after delivery.
- COD amount is added to rider cash ledger.

12. Notifications

The backend sends FCM push notifications through Firebase Admin.

Token registration:

- Apps register notification token to /notifications/register-token.
- Tokens include app type: customer, partner, or delivery.
- Tokens include platform and optional device ID.
- Logout or explicit unregister can disable/remove tokens.
- Invalid FCM tokens are disabled after failed sends.

Notification examples:

- Partner new order.
- Customer order status updates.
- Delivery job ready.
- Delivery assigned.
- Delivery partner assigned by admin.
- Partner delivery status updates.
- Partner application approved/rejected/reupload requested.
- Delivery application approved/rejected/reupload requested.
- Payment confirmed.

Android notifications use app-specific colors and a shared notification channel concept configured by custom Expo plugin.

13. Uploads and Documents

Uploads flow through the backend rather than directly from apps to Cloudinary.

Upload endpoint:

- /api/upload/image
- Auth required before parsing multipart data.
- Accepts image or file.
- Allows JPG, JPEG, PNG, WEBP, HEIC, HEIF, PDF, and some image MIME variants.
- Max route upload size is 15 MB.
- Cloudinary upload timeout is 180 seconds.
- Retry is attempted for retryable Cloudinary/network errors.
- Images are transformed to max 800x800.
- Files are stored under a user-specific Cloudinary folder.

Deletion endpoint:

- /api/upload/image
- Non-admin users can only delete files uploaded under their own user folder.
- Admin can delete more broadly.

Upload users:

- Partner onboarding documents.
- Partner profile, shop logo, banner, restaurant photos, menu images.
- Delivery profile/document uploads.
- Cash deposit proof may reference uploaded proof URL.

14. Admin and Operations

The admin panel is the operations control center.

Dashboard:

- Shows total orders.
- Shows pending/today orders.
- Shows partner counts.
- Shows active/pending partners.
- Shows earnings metrics.

Partner review:

- Admin can list partner applications.
- Admin can view partner details.
- Admin can approve, reject, or suspend.
- Admin can request specific document reuploads with notes.
- Partner app surfaces reupload flags and notes.

Delivery review:

- Admin can list delivery partners.
- Admin can verify, activate, reject, suspend, or mark inactive depending UI/backend use.
- Admin can request delivery document reuploads with notes.
- Delivery app routes users based on status.

Order operations:

- Admin can view all orders.
- Admin can view details.
- Admin can update supported statuses.
- Admin can manually assign delivery partner when order is ready.

Support:

- Customers create tickets and add messages.
- Admin lists tickets by status.
- Admin replies and can update ticket status.

Payouts:

- Admin generates weekly or monthly summaries.
- Partner payout amount is currently based on delivered order item totals.
- Delivery payout amount is based on delivery fees.
- Delivery payout can be offset by cash held from COD.
- Admin creates paid payout records.
- Orders are marked with partner/delivery payout status.

Cash deposits:

- Rider submits deposit request.
- Admin verifies or rejects.
- Verification creates a posted ledger entry and reduces rider cash balance.
- Rejection removes pending deposit amount but does not reduce cash balance.

15. Security and Reliability Features

Security and reliability already present:

- JWT auth middleware on protected routes.
- Role middleware on admin/customer-specific routes.
- Session invalidation through sessionVersion.
- Production environment validation.
- Admin password login requires explicit env configuration.
- OTP rate limits for sending and verification.
- Admin login rate limit.
- General API rate limiting.
- CORS origin restrictions in production.

- Helmet enabled.
- Razorpay checkout signature verification.
- Razorpay webhook signature verification.
- Cloudinary delete ownership checks.
- Upload type restrictions.
- Auth before multipart parsing.
- Account deletion and logout invalidate sessions.
- Firebase token verification checks expected phone number.
- FCM invalid tokens are disabled.
- Order status transitions are guarded by role and state.
- Delivery one-active-job guard.
- COD delivery requires collected amount.
- Stale order auto-cancellation sweep.

Operational reliability:

- Mobile API clients support production and local fallback URLs.
- Customer app uses longer API timeout because hosted backend may wake slowly.
- Mobile clients refresh expired access tokens.
- Partner app proactively refreshes tokens before expiry.
- Startup route checks avoid forcing logout on transient network failures in several apps.

16. Build and Run Commands

Customer app:

- npm start
- npm run android
- npm run ios
- npm run web
- npm run typecheck
- npm run build:android
- npm run submit:android

Partner app:

- npm start
- npm run android
- npm run ios
- npm run web
- npm run typecheck
- npm run build:android
- npm run submit:android

Delivery app:

- npm start

- npm run android
- npm run ios
- npm run web
- npm run typecheck
- npm run build:android
- npm run submit:android

Admin panel:

- npm run dev
- npm run build
- npm run lint
- npm run preview

Public website:

- npm run dev
- npm run build
- npm run lint
- npm run preview

Backend:

- npm run dev
- npm run build
- npm start
- npm test
- npm run mongo:dev
- npm run seed:partner-mock-menus

17. Important Environment Variables

Backend environment variables include:

- PORT
- NODE_ENV
- MONGODB_URI or MONGO_URI
- JWT_SECRET
- JWT_EXPIRY
- REFRESH_JWT_EXPIRY
- API_BASE_URL
- CORS_ORIGINS
- REQUEST_BODY_LIMIT
- RATE_LIMIT_WINDOW_MS
- RATE_LIMIT_MAX
- OTP_PROVIDER
- OTP_EXPIRY_MINUTES
- OTP_RESEND_COOLDOWN_SECONDS

- OTP_MAX_ATTEMPTS
- TWILIO_ACCOUNT_SID
- TWILIO_AUTH_TOKEN
- TWILIO_VERIFY_SERVICE_SID
- MSG91_AUTH_KEY
- MSG91_TEMPLATE_ID
- MSG91_SENDER_ID
- FIREBASE_PROJECT_ID
- FIREBASE_SERVICE_ACCOUNT_PATH
- FIREBASE_SERVICE_ACCOUNT_JSON
- ADMIN_PANEL_PHONE
- ADMIN_PANEL_PASSWORD
- RAZORPAY_KEY_ID
- RAZORPAY_KEY_SECRET
- RAZORPAY_WEBHOOK_SECRET
- CLOUDINARY_CLOUD_NAME
- CLOUDINARY_API_KEY
- CLOUDINARY_API_SECRET
- CLOUDINARY_UPLOAD_FOLDER
- GOOGLE_MAPS_API_KEY
- DELIVERY_RADIUS_KM
- DELIVERY_LOCATION_FRESHNESS_MINUTES

Mobile apps use:

- EXPO_PUBLIC_API_URL when a specific API base URL is needed.
- Firebase config through google-services.json.
- Expo update/EAS project IDs in app.json.

Admin panel uses:

- VITE_API_URL for backend API override.

18. Order Status Reference

Customer-facing order path:

- PENDING: online payment not completed yet.
- CONFIRMED: order is placed and waiting for partner.
- ACCEPTED: partner accepted.
- PREPARING: partner is preparing.
- READY: order is ready for pickup.
- ASSIGNED: delivery partner assigned/accepted.
- PICKED_UP: delivery partner picked up.
- DELIVERED: order completed.
- CANCELLED: cancelled by timeout, customer, admin, or system condition.
- REJECTED: partner rejected; treated as cancellation from customer perspective.

Payment statuses:

- PENDING: online payment pending.
- PAYMENT_PENDING_DELIVERY: COD pending until delivery.
- PAID: paid online or COD collected.
- FAILED: online payment failed.
- REFUNDED: refund expected/recorded after cancellation.
- CANCELLED: payment/order cancelled before payment completion.

Partner statuses:

- PENDING
- APPROVED
- REJECTED
- SUSPENDED

Delivery partner statuses:

- PENDING
- VERIFIED
- ACTIVE
- REJECTED
- SUSPENDED
- INACTIVE

19. End-to-End Example

A normal prepaid order works like this:

1. Customer logs in.
2. Customer completes profile and address pin.
3. Customer sees nearby approved shops.
4. Customer selects a shop and adds available menu items.
5. Cart asks backend for pricing.
6. Customer chooses Razorpay.
7. Backend creates PENDING order.
8. Backend creates Razorpay checkout order.
9. Customer pays in Razorpay.
10. Backend verifies payment signature.
11. Backend marks payment PAID and order CONFIRMED.
12. Partner gets push notification.
13. Partner accepts order.
14. Partner marks preparing.
15. Partner marks ready.
16. Delivery partners get ready job notification.
17. Rider accepts job.
18. Customer and partner get delivery assigned notification.

19. Rider marks picked up.
20. Customer and partner are notified.
21. Rider marks delivered.
22. Backend marks order delivered.
23. Rider earnings increase by delivery fee.
24. Order becomes eligible for partner and delivery payout summary.

A COD order differs only in payment timing:

- Order starts as CONFIRMED.
- Payment status is PAYMENT_PENDING_DELIVERY.
- Rider collects cash at delivery.
- Backend marks payment PAID.
- Cash is added to rider cash balance.
- Admin later reconciles cash deposits and payout offsets.

20. QA and Store Readiness

The repository includes QA and Play Store readiness documents.

Important customer QA areas:

- Fresh install login and OTP.
- Mandatory profile/address completion.
- Shop list and menu loading.
- Cart quantity and checkout totals.
- COD order creation.
- Razorpay payment success/failure/cancel flows.
- Order status updates.
- Logout and account deletion.

Important partner QA areas:

- OTP login.
- Onboarding validation.
- Pending/rejected/approved screens.
- Shop status/profile/hours/image updates.
- Menu CRUD and availability toggle.
- Order accept/reject/preparing/ready flow.
- Logout and account deletion.

Important delivery QA areas:

- OTP login.
- Profile/document onboarding.
- Location permission denial and approval.
- Nearby jobs.
- Accept, pickup, deliver.

- COD amount prompt.
- Earnings and cash ledger.
- Logout and account deletion.

Important admin/backend QA areas:

- Health endpoint.
- Public legal endpoints.
- Production CORS.
- Admin login.
- Dashboard stats.
- Partner approval/rejection/reupload.
- Delivery verification/rejection/reupload.
- Order list/details/status/assignment.
- Razorpay signature and webhook verification.
- Upload ownership protection.
- Account deletion/session invalidation.

21. Current System Strengths

- Clear separation between customer, partner, delivery, admin, backend, and website.
- Role-aware backend with shared authentication.
- Real business workflows are represented end-to-end.
- Mobile apps include production API fallback.
- Payments are signature verified.
- COD cash handling is modeled with ledger entries.
- Payouts are modeled separately from orders.
- Partner and delivery document reupload is supported.
- Public legal and Play Store readiness content exists.
- Stale order cancellation is automated.
- Push notifications cover the important operational moments.

22. Areas to Watch

These are not necessarily bugs, but they are important product/engineering areas to keep in mind:

- SubOrder model exists, but current visible routes mainly use direct Order flows.
Confirm whether suborders are legacy or planned.
- Socket.IO is installed in backend dependencies but the visible server currently uses normal Express listen without a Socket.IO server setup. Current real-time behavior is primarily polling/push notifications unless implemented elsewhere later.
- Some enum values such as card, UPI, and wallet exist in the order model, while the customer UI mainly supports Razorpay and COD.

- Several apps include fallback behavior on transient network errors. This improves user experience but should be tested so stale sessions do not hide real authorization failures.
- Production OTP must not use memory provider; backend already throws in production if memory provider is attempted, but environment setup must be correct.
- Payout math is currently based on item total for partners and delivery fee for riders. If commission, tax, service fee, refund, or penalty logic changes, payout calculations need to be updated.
- Delivery location freshness and radius configuration should be reviewed before launch for actual city density.
- Public website policies are strong, but should be reviewed by a qualified legal advisor before production launch.

23. Short Summary

Vyaha/NearU is a full hyperlocal delivery platform. Customers order from nearby approved shops, partners manage their restaurant/shop operations, riders fulfill ready delivery jobs, admins control verification/support/payouts, and the backend coordinates authentication, orders, payments, notifications, uploads, and data persistence. The project is already structured like a production ecosystem rather than a prototype, with separate apps for each role and explicit operational workflows for onboarding, ordering, delivery, support, and payouts.